Creating an environment from a file

Just as the configuration of an environment can be copied to a file, it is possible for users to create an environment file themselves. The Conda environment file may contain the following records: name, channels and dependencies. To create an environment using a *yml* file, the file should contain the name of the environment at least. *Channels* contain the list of paths or URLs of repositories where Conda should look for the packages to be installed. 'defaults' in the channel list indicates that Conda should search in the default repositories while installing packages. Users are allowed to give their priorities for selecting channels. The following code shows a configuration file, in which it is specified that *numpy* must be installed from the Anaconda repository:

```
name: e1
channels:
- https://anaconda.org/conda-forge/numpy
dependencies:
- numpy
```

Dependencies list the packages to be installed in the environment. It is okay if we do not know the dependencies of a particular package. Conda, while creating an environment from the file, will check for the dependencies of the packages specified in the file and resolve them.

Configuration file

The *.condarc* file is generated by the command:

```
conda config
```

This configuration file will help advanced users to set their preferences for *Channels*, configure proxy servers, set package managers, and much more.

Thus, Conda that comes with Anaconda or Miniconda opens up the magic world of packages for data science applications. It is a powerful alternative to many package managers as well as environment managers. END

References

[1] *https://conda.io*
[2] *http://docs.ansible.com/ansible/YAMLSyntax.html*

By: Sharon Sunny

The author is an assistant professor at Amaljyothi College of Engineering, Kerala. She can be reached at *ssharon099@gmail.com*.

Continued from Page 37...

6. Now the coding is complete for your first JavaFX application. Run your project and you will see the window shown in Figure 3.
7. Check your application by typing some text and clicking the button. The output will be printed on the console.

Having coded your application and successfully run it, let's take a look at the code and how it works.

Every JavaFX application must extend the *Application* class as done above, and will consist of a *Stage* which will contain *Scene*, which will, in turn, contain *Layout*. *Layout* will contain all the widgets or controls like buttons, text fields, etc, of your application.

- In the above code, in the *Main* method, we have called the *Launch* method, which will launch your application. In the overridden *Start* method, we have designed our GUI.
- First, we have defined two controls—one text field and one button. To the button, we have added an event handler, which performs the action when we click the button.
- Then we have defined a layout, which is *BorderPane* in this case; next, we have defined and added *VBox* to our border pane (through the *pane.setCenter()* method), which will help us to keep our controls vertically below each other.
- Then, we have added controls to our *VBox* (through the *getChildren()* method).
- We have then defined *Scene* as explained earlier and added our *BorderPane* to it. Finally, we have added *Scene* to the *Stage*, and then we have displayed the *Stage* using the *show()* method.

This concludes our tutorial for a basic JavaFX GUI application development through Eclipse. I hope you have got a fair idea about the different components of JavaFX and how to implement them in application development. If you have any doubts, you can reach me at my email. END

By: Vinayak Vaid

The author works as an automation engineer at Infosys Limited, Pune. He has worked on different testing technologies and automation tools like QTP, Selenium and Coded UI. He can be contacted at *vinayakvaid91@gmail.com*.